

Web API & RESTful Web Services

1. RESTful Web Service, Web API & Microservice

1.1 What is a Web Service?

A Web Service is a software system designed to support interoperable machine-to-machine interaction over a network. It allows different applications to communicate with each other regardless of the programming language or platform they use.

1.2 What is a Web API?

A Web API (Application Programming Interface) is an interface that allows communication between different software applications over HTTP. Unlike traditional Web Services, Web APIs are not restricted to XML and can return data in multiple formats like JSON, XML, plain text, etc.

1.3 Difference Between Web Service and Web API

Feature	Web Service	Web API
Protocol	SOAP, REST, XML-RPC	HTTP/HTTPS only
Data Format	Only XML	JSON, XML, plain text, etc.
Platform	Platform dependent	Platform independent
Performance	Slower due to XML	Faster due to JSON support
Flexibility	Less flexible	More flexible
Usage	Enterprise applications	Mobile & Web applications
Standard	WS-* standards	No strict standard

1.4 Features of REST Architecture

REST stands for Representational State Transfer. It is an architectural style for designing networked applications.

- Representational State Transfer - Resources are represented in formats like JSON or XML
- Stateless - Each request from client to server must contain all information needed. Server does not store session state
- Client-Server - Separation of concerns between UI and data storage
- Cacheable - Responses can be cached to improve performance
- Uniform Interface - Consistent way to interact with resources using HTTP verbs
- Layered System - Client cannot tell whether it is connected directly to the end server

1.5 What is a Microservice?

A Microservice is an architectural approach where an application is built as a collection of small, independent services that communicate through APIs.

- Each service runs independently and can be deployed separately
- Services communicate via REST APIs or message queues
- Each service handles a specific business function
- Easier to scale individual services based on demand
- Technology independent - each service can use different tech stack

2. HttpRequest & HttpResponse

2.1 HttpRequest

An HTTP Request is a message sent by the client to the server to request data or perform an action. It consists of:

- Request Line - HTTP method, URL, and HTTP version
- Headers - Metadata like Content-Type, Authorization, Accept
- Body - Data sent with POST/PUT requests

```
Example HTTP Request:  
POST /api/products HTTP/1.1  
Host: localhost:5000  
Content-Type: application/json  
Authorization: Bearer token123
```

```
{  
  "Name": "Laptop",  
  "Price": 75000  
}
```

2.2 HttpResponse

An HTTP Response is the message sent by the server back to the client after processing the request. It consists of:

- Status Line - HTTP version and status code
- Headers - Metadata like Content-Type, Cache-Control
- Body - The actual data returned (JSON, XML, HTML, etc.)

Example HTTP Response:
HTTP/1.1 200 OK
Content-Type: application/json

```
{  
  "Id": 1,  
  "Name": "Laptop",  
  "Price": 75000  
}
```

3. Types of Action Verbs (HTTP Methods)

HTTP verbs define the type of action to be performed on the resource. In Web API, these are declared as attributes on action methods.

HTTP Verb	Attribute	Purpose	Example URL
GET	[HttpGet]	Retrieve data from server	GET /api/products
POST	[HttpPost]	Create new resource on server	POST /api/products
PUT	[HttpPut]	Update existing resource completely	PUT /api/products/1
DELETE	[HttpDelete]	Delete existing resource	DELETE /api/products/1
PATCH	[HttpPatch]	Partially update a resource	PATCH /api/products/1

3.1 How Action Verbs are Declared in Web API

```
[ApiController]  
[Route("api/[controller]")]  
public class ProductsController : ControllerBase  
{  
    // HttpGet - Read all products  
    [HttpGet]  
    public ActionResult<IEnumerable<Product>> Get() { }  
  
    // HttpGet with ID - Read single product  
    [HttpGet("{id}")]  
    public ActionResult<Product> Get(int id) { }  
  
    // HttpPost - Create new product  
    [HttpPost]  
    public ActionResult Post([FromBody] Product product) { }  
  
    // HttpPut - Update existing product
```

```

[HttpPut("{id}")]
public ActionResult Put(int id, [FromBody] Product product) { }

// HttpDelete - Delete product
[HttpDelete("{id}")]
public ActionResult Delete(int id) { }
}

```

4. HTTP Status Codes in Web API

HTTP Status Codes are standard response codes that indicate the result of an HTTP request.

Status Code	Name	ActionResult Type	When to Use
200	OK	return Ok(data)	Request succeeded, data returned
201	Created	return Created(url, data)	New resource created successfully
204	No Content	return NoContent()	Success but no data to return
400	Bad Request	return BadRequest(msg)	Invalid input or validation error
401	Unauthorized	return Unauthorized()	Authentication required
403	Forbidden	return Forbid()	Access denied even with auth
404	Not Found	return NotFound(msg)	Resource does not exist
500	Internal Server Error	return StatusCode(500)	Server side error occurred

4.1 Example of Status Codes in Action Methods

```

[HttpGet("{id}")]
public ActionResult<Product> Get(int id)
{
    var product = _products.Find(p => p.Id == id);

    if (product == null)
        return NotFound("Product not found!");    // 404

    return Ok(product);                            // 200
}

[HttpPost]
public ActionResult Post([FromBody] Product product)
{

```

```

        if (!ModelState.IsValid)
            return BadRequest(ModelState); // 400

        _products.Add(product);
        return Created("/api/products", product); // 201
    }

```

5. Structure of a Web API

5.1 Controller Structure

A Web API Controller is a class that handles HTTP requests. It inherits from ControllerBase (in .NET Core) or ApiController (in .NET 4.5).

```

using Microsoft.AspNetCore.Mvc;

namespace FirstWebAPI.Controllers
{
    [ApiController] // Marks as API controller
    [Route("api/[controller]")] // Sets route to api/values
    public class ValuesController : ControllerBase // Inherits
        ControllerBase
    {
        private static List<string> values = new List<string>
        {
            "Value1", "Value2", "Value3"
        };

        // Action Verb: GET
        // Action Method: Get()
        [HttpGet]
        public ActionResult<IEnumerable<string>> Get()
        {
            return Ok(values);
        }

        [HttpGet("{id}")]
        public ActionResult<string> Get(int id)
        {
            if (id < 0 || id >= values.Count)
                return NotFound("Value not found!");
            return Ok(values[id]);
        }

        [HttpPost]
        public ActionResult Post([FromBody] string value)
        {
            values.Add(value);
            return Ok("Value added!");
        }
    }

```

```

        [HttpPut("{id}")]
        public ActionResult Put(int id, [FromBody] string value)
        {
            values[id] = value;
            return Ok("Value updated!");
        }

        [HttpDelete("{id}")]
        public ActionResult Delete(int id)
        {
            values.RemoveAt(id);
            return Ok("Value deleted!");
        }
    }
}

```

5.2 Key Components of Web API Structure

Component	Description	Example
Controller	Class that handles HTTP requests	ValuesController
Inheritance	Must inherit from ControllerBase	: ControllerBase
[ApiController]	Marks class as Web API controller	[ApiController]
[Route]	Defines URL pattern for controller	[Route("api/[controller]")]
Action Verb	Defines HTTP method for action	[HttpGet], [HttpPost]
Action Method	Method that handles the request	public ActionResult Get()
ActionResult	Return type of action methods	Ok(), NotFound(), BadRequest()

6. Configuration Files of Web API

6.1 Program.cs / Startup.cs (.NET Core)

In .NET Core, Program.cs is the entry point. It configures services and middleware using dependency injection.

```

// Program.cs (.NET 6/7/8)
var builder = WebApplication.CreateBuilder(args);

```

```
// Add services - Dependency Injection
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// Configure middleware pipeline
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

6.2 appsettings.json

Stores application configuration settings like connection strings, logging, and custom settings.

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=localhost;Database=RetailDB;Trusted_Connection=True;"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

6.3 launchSettings.json

Configures how the application is launched during development. Located in Properties folder.

```
{
  "profiles": {
    "http": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "launchUrl": "swagger",
      "applicationUrl": "http://localhost:5000",
      "environmentVariables": {
```

```

        "ASPNETCORE_ENVIRONMENT": "Development"
    }
}
}
}

```

6.4 Route.config & WebAPI.config (.NET 4.5 Framework)

In .NET Framework 4.5, routing and Web API configuration were done in separate files:

- RouteConfig.cs - Configures MVC routes for the application
- WebApiConfig.cs - Configures Web API specific routes and formatters

```

// WebApiConfig.cs (.NET 4.5)
public static class WebApiConfig
{
    public static void Register(HttpConfiguration config)
    {
        // Web API routes
        config.MapHttpAttributeRoutes();

        config.Routes.MapHttpRoute(
            name: "DefaultApi",
            routeTemplate: "api/{controller}/{id}",
            defaults: new { id = RouteParameter.Optional }
        );
    }
}

```

6.5 Comparison: .NET Core vs .NET 4.5 Configuration

Config File	.NET Core	.NET Framework 4.5
Entry Point	Program.cs	Global.asax.cs
Service Config	Program.cs (DI)	Startup.cs / App_Start/
API Routes	Attribute routing in Program.cs	WebApiConfig.cs
MVC Routes	Attribute routing	RouteConfig.cs
App Settings	appsettings.json	Web.config
Launch Settings	launchSettings.json	IIS Express settings
Controller Base	ControllerBase	ApiController